

PetAgenda

Sistema Web de Agendamento para Pet Shop

Documentação do Projeto

Integrantes:

Anttonio Osório Molinaro Maccagnini

Gabriel Lengert Guedes

Turma: T2ESOFT05NB

Disciplina: Desenvolvimento Web II

Centro Universitário Católica de Santa Catarina — Joinville

Junho de 2025

1. Identificação do Projeto

Nome do projeto: PetAgenda

Descrição: Sistema web de agendamento para pet shop

Integrantes: Anttonio Osório Molinaro Maccagnini e Gabriel Lengert Guedes

Turma: T2ESOFT05NB

Disciplina: Programação Web

Instituição: Centro Universitário Católica de Santa Catarina — Joinville

Repositório: https://github.com/anttonio06/Desenvolvimento_web2

Aplicação em produção: <https://desenvolvimento-web2.onrender.com>

Protótipo (Figma):

<https://www.figma.com/design/4812XawUkC89akWzElnj8Q/Projeto-Pet-Shop>

2. Descrição Geral do Sistema

O PetAgenda é uma aplicação web desenvolvida para auxiliar na organização dos atendimentos em pet shops. O sistema reúne em um único ambiente o cadastro de clientes, seus respectivos pets, os serviços oferecidos e os agendamentos realizados.

A proposta do projeto surgiu para reduzir problemas comuns de controles feitos manualmente, como conflitos de horário e dificuldades na organização dos atendimentos. Durante a criação de um agendamento, o sistema verifica a disponibilidade do horário, identifica possíveis duplicidades envolvendo o mesmo pet e serviço e aplica algumas regras entre os serviços para evitar inconsistências.

2.1 Objetivos

- Desenvolver um sistema web com renderização no servidor (SSR), utilizando Node.js e EJS como motor de templates;
- Utilizar o SQLite como banco de dados relacional, aplicando restrições de integridade e transações para garantir a consistência das informações;
- Implementar operações completas de cadastro, consulta, edição e exclusão (CRUD) para as entidades do sistema;
- Desenvolver o processo de agendamento com as validações necessárias para atender às regras de negócio definidas;
- Implementar autenticação de usuários com níveis de acesso diferentes de acordo com o perfil de cada usuário.

3. Tecnologias Utilizadas

Tecnologia / Biblioteca	Finalidade
Node.js	Ambiente de execução do servidor
Express	Framework HTTP e roteamento
EJS	Motor de templates utilizado para renderizar as páginas no servidor
express-ejs-layouts	Layout compartilhado entre as views
SQLite3	Banco de dados relacional embutido
connect-sqlite3	Armazenamento das sessões de usuário no SQLite
express-session	Gerenciamento de sessões HTTP
bcrypt	Criptografia das senhas utilizando hash
dotenv	Gerenciamento de variáveis de ambiente
Bootstrap	Framework CSS para interface responsiva
Jest	Framework de testes automatizados
Nodemon	Reinício automático do servidor em desenvolvimento
Render	Plataforma de hospedagem em nuvem (produção)
GitHub Actions	Automação do processo de integração contínua e deploy (CI/CD)

4. Arquitetura do Sistema

O projeto foi organizado seguindo o padrão MVC (Model-View-Controller), com as páginas sendo renderizadas no servidor. A estrutura foi dividida em camadas para separar melhor as responsabilidades de cada parte da aplicação. A tabela abaixo apresenta a função de cada componente e sua localização no projeto.

Camada	Responsabilidade	Localização
Routes (Controller)	Recebe as requisições, realiza os processamentos necessários e retorna a resposta ao usuário	/routes/*.routes.js
Views	Responsáveis pela geração das páginas HTML utilizando templates EJS	/views/**/*.ejs
Database	Camada responsável pelas operações no banco de dados SQLite e pelo gerenciamento das transações	/database/
Middlewares	Realizam verificações relacionadas à autenticação e às permissões de acesso	/middlewares/
Utils / Helpers	Funções auxiliares utilizadas em validações, formatações e cálculos do sistema	/utils/helpers.js
Public	Arquivos estáticos, como folhas de estilo e imagens	/public/

4.1 Fluxo de uma Requisição

O processamento de uma requisição ocorre da seguinte maneira:

- O usuário acessa uma determinada rota pelo navegador;
- O Express verifica se existem validações de autenticação ou permissões associadas à rota;
- A rota executa as operações necessárias e realiza consultas no banco de dados quando preciso;
- Os dados obtidos são enviados para a view correspondente;
- A view gera o HTML da página, que é retornado ao navegador do usuário.

5. Modelo de Dados

Tabela	Colunas Principais
usuarios	id, nome, email, senha (hash bcrypt), senha_texto, permissoes, reset_token, reset_token_expiry
clientes	id, nome, telefone (unique), email (unique), criado_em
pets	id, nome, especie, raca, porte, cliente_id (FK)
servicos	id, nome, duracao_min, preco_pequeno, preco_medio, preco_grande
funcionarios	id, nome, telefone, cpf, email, ativo, usuario_id (FK), criado_em
agendamentos	id, cliente_id (FK), pet_id (FK), data_hora, status, observacoes, valor, desconto, funcionario_id (FK)
agendamento_servicos	id, agendamento_id (FK), servico_id (FK), valor

Algumas restrições implementadas no banco de dados:

- Os campos de e-mail e telefone dos clientes utilizam índices UNIQUE parciais, permitindo ignorar valores nulos ou vazios;
- A tabela agendamento_servicos possui exclusão em cascata vinculada ao agendamento correspondente, evitando registros órfãos quando um agendamento é removido;
- A senha utilizada na autenticação é armazenada na coluna senha utilizando hash bcrypt com 10 rounds. Além disso, existe a coluna senha_texto, que mantém a senha em texto puro para exibição ao administrador na tela de funcionários. Essa solução foi utilizada durante o desenvolvimento para facilitar o gerenciamento dos funcionários, mas representa um aspecto que deveria ser revisto em versões futuras por questões de segurança..

6. Funcionalidades Implementadas

6.1 Autenticação e Controle de Acesso

O sistema trabalha com dois tipos de usuários: administrador e funcionário. O acesso é realizado por meio de e-mail e senha, sendo utilizada a verificação do hash armazenado com bcrypt. Após o login, as informações da sessão são armazenadas no banco de dados e permanecem válidas por até duas horas. Ao realizar logout, a sessão é encerrada.

As rotas internas são protegidas por um middleware de autenticação, enquanto funcionalidades exclusivas do administrador contam com uma verificação adicional de permissão. O processo de recuperação de senha utiliza um token gerado pela

biblioteca nativa crypto do Node.js, com validade de uma hora. O sistema não realiza envio de e-mails; por isso, a redefinição é feita diretamente por meio da rota /redefinir-senha/:token. O cadastro de novos usuários é restrito ao administrador e é realizado pelo módulo de funcionários.

6.2 Gestão de Clientes

O módulo de clientes permite cadastrar, editar, listar e excluir registros. Durante o cadastro, o sistema verifica se o formato do telefone e do e-mail é válido e impede a duplicidade dessas informações. A listagem apresenta os clientes juntamente com seus respectivos pets. Quando um cliente é removido, seus pets e os agendamentos relacionados também são excluídos por meio das regras definidas no banco de dados.

6.3 Gestão de Pets

Cada pet é associado a um cliente e possui informações como espécie, raça e porte (Pequeno, Médio ou Grande). O porte do animal é utilizado no cálculo dos valores dos serviços durante os agendamentos. Caso um pet seja removido, os agendamentos vinculados a ele também são excluídos.

6.4 Gestão de Serviços

O sistema possui cinco serviços cadastrados na inicialização do banco de dados: Banho, Tosa, Hidratação, Consulta Básica e Corte de Unhas. Não existe uma funcionalidade para cadastrar ou remover serviços pela interface. O administrador pode apenas alterar os preços de acordo com o porte do animal. A duração de cada serviço também é armazenada no banco, sendo definida durante a configuração inicial do sistema.

6.5 Gestão de Funcionários

O gerenciamento de funcionários é uma funcionalidade exclusiva do administrador. Ao cadastrar um funcionário, o sistema cria os registros necessários nas tabelas de usuários e funcionários, estabelecendo a relação entre elas. Também é possível ativar ou desativar funcionários. Usuários inativos não conseguem realizar login, mesmo informando a senha correta. Caso um funcionário seja removido, os agendamentos antigos permanecem armazenados no sistema.

6.6 Gestão de Agendamentos

O módulo de agendamentos concentra a principal funcionalidade da aplicação. Durante a criação de um agendamento, o usuário escolhe o pet, a data, o horário e os serviços desejados, podendo também informar o funcionário responsável. O valor total é calculado automaticamente de acordo com o porte do animal e os serviços selecionados.

Antes da confirmação, são realizadas as validações descritas na seção 7. A listagem dos agendamentos é paginada, exibindo 20 registros por página, e permite filtros por status. O sistema também disponibiliza um endpoint utilizado pela visualização em calendário.

6.7 Dashboard

Após o login, o usuário é direcionado para uma página inicial com informações gerais do sistema. Nela são exibidos indicadores como quantidade de clientes, pets, serviços cadastrados e agendamentos previstos para o dia.

Além desses dados, o dashboard apresenta os clientes cadastrados mais recentemente, os próximos agendamentos ativos, os últimos pets adicionados ao sistema e a lista de funcionários que estão em atividade.

7. Regras de Negócio

Regra	Descrição
Capacidade máxima por horário	O sistema permite no máximo três agendamentos por hora. Quando esse limite é atingido, o horário deixa de ficar disponível para novos agendamentos.
Duplicidade de pet e serviço	Um mesmo pet não pode possuir o mesmo serviço agendado para a mesma data e horário. Agendamentos cancelados não são considerados nessa verificação.
Dependência entre serviços	Os serviços de Tosa e Hidratação só podem ser realizados em conjunto com o serviço de Banho. Caso essa condição não seja atendida, o sistema impede a criação do agendamento.
Proibição de datas passadas	Não é permitido criar agendamentos com datas anteriores à data atual.
Restrição por perfil	Apenas usuários com perfil de administrador possuem acesso ao módulo de funcionários e às informações restritas do sistema.
Armazenamento de senhas	A senha utilizada na autenticação é armazenada utilizando hash bcrypt (10 rounds). Além disso, a coluna senha_texto mantém a senha original para exibição no módulo de funcionários.
Bloqueio de usuário inativo	Funcionários marcados como inativos não conseguem acessar o sistema, mesmo informando credenciais válidas.
Transações	Operações como criação e edição de agendamentos, além de exclusões relacionadas, são executadas dentro

Regra	Descrição
	de transações do SQLite, evitando inconsistências em caso de falha durante o processo.

8. Requisitos do Sistema

8.1 Requisitos Funcionais

ID	Descrição
RF01	O sistema deve permitir login e logout de usuários autenticados.
RF02	O sistema deve bloquear o login de funcionários com status inativo.
RF03	O sistema deve permitir recuperação de senha via token com validade de uma hora, acessível diretamente por URL — sem envio de e-mail.
RF04	O sistema deve permitir o cadastro, edição, listagem e exclusão de clientes.
RF05	O sistema deve validar unicidade de e-mail e telefone no cadastro de clientes.
RF06	O sistema deve permitir o cadastro, edição e exclusão de pets vinculados a clientes.
RF07	O sistema deve excluir agendamentos em cascata ao excluir um pet ou cliente.
RF08	O sistema deve permitir a edição dos preços dos serviços por porte de animal.
RF09	O sistema deve permitir o cadastro e gerenciamento de funcionários, restrito ao administrador.
RF10	O sistema deve permitir criar agendamentos com um ou mais serviços selecionados.
RF11	O sistema deve calcular automaticamente o valor total do agendamento com base no porte do pet.
RF12	O sistema deve validar conflitos de horário, duplicidade e dependência de serviços antes de confirmar um agendamento.
RF13	O sistema deve permitir alterar o status do agendamento para concluído ou cancelado.
RF14	O sistema deve exibir um dashboard com totalizadores e resumo das operações do dia.
RF15	O sistema deve disponibilizar um endpoint de calendário com agendamentos filtrados por período.

8.2 Requisitos Não Funcionais

ID	Descrição
RNF01	O sistema deve renderizar todas as páginas no servidor (SSR), sem depender de frameworks JavaScript no lado do cliente.
RNF02	As senhas utilizadas na autenticação devem ser armazenadas utilizando hash bcrypt com no mínimo 10 rounds.
RNF03	Todas as rotas privadas devem possuir proteção por meio de middleware de autenticação.
RNF04	O sistema deve utilizar um banco de dados relacional com restrições de integridade referencial.
RNF05	O pipeline de CI/CD deve executar os testes automatizados e o smoke test antes da realização do deploy.
RNF06	As sessões dos usuários devem expirar automaticamente após duas horas de inatividade.
RNF07	A interface do sistema deve ser responsiva e desenvolvida com Bootstrap 5.
RNF08	O banco responsável pelo armazenamento das sessões deve permanecer separado do banco principal da aplicação.
RNF09	Operações críticas devem ser executadas dentro de transações, evitando inconsistências em caso de falhas durante a execução.
RNF10	O sistema deve estar disponível em produção por meio de uma plataforma em nuvem com processo de deploy automatizado.

9. Pipeline de CI/CD

O projeto utiliza GitHub Actions para automatizar o processo de testes e deploy. A configuração está definida no arquivo `.github/workflows/ci-cd.yml`.

O pipeline é executado sempre que ocorre um push ou um pull request para a branch main. Ele é dividido em duas etapas principais: CI (Build & Test) e Deploy.

Durante a etapa de CI, o pipeline executa as seguintes tarefas:

Etapas	Descrição
Checkout do código	Clona o repositório em uma máquina virtual do GitHub Actions
Configurar Node.js 20	Instala e configura o ambiente Node.js 20

Etapa	Descrição
Instalar dependências	Executa npm install para instalar as dependências do projeto
Verificar sintaxe	Executa node --check app.js para verificar a sintaxe do arquivo principal
Inicializar banco de dados	Executa npm run init-db, responsável pela criação das tabelas e inserção dos dados iniciais
Executar testes	Executa os testes automatizados com Jest (npm test)
Smoke test	Inicia o servidor temporariamente e verifica, por meio do curl, se a rota /login responde corretamente

A etapa de Deploy é executada somente quando a etapa de CI é concluída com sucesso e o evento corresponde a um push realizado diretamente na branch main. Dessa forma, pull requests não realizam deploy automaticamente.

A publicação em produção é feita por meio de um webhook da plataforma Render, armazenado como variável de ambiente secreta no repositório (RENDER_DEPLOY_HOOK_URL).

10. Testes Automatizados

O projeto possui testes automatizados desenvolvidos com Jest, localizados em testes/validacoes.test.js. Os testes foram desenvolvidos para validar as principais funções presentes em helpers.js, utilizadas em diferentes partes da aplicação.

Ao todo, foram implementados 24 casos de teste, distribuídos entre os grupos apresentados na tabela abaixo:

Grupo	Casos de Teste
capitalize	Testa a conversão da primeira letra de cada palavra para maiúscula, a normalização de textos em caixa alta e a remoção de espaços extras nas extremidades da string.
telPattern	Verifica se números de telefone fixos e celulares com DDD são aceitos e se entradas inválidas, como letras ou strings vazias, são rejeitadas.
emailPattern	Avalia a validação de e-mails com domínio .com e .com.br, além da rejeição de formatos inválidos ou extensões não suportadas, como .xyz.
normalizarServicos	Verifica o comportamento da função para valores null e undefined, a conversão de strings simples em arrays e a manutenção de arrays já existentes.

Grupo	Casos de Teste
calcularValores	Testa o cálculo do valor final de acordo com o porte do pet, além do comportamento adotado para portes não reconhecidos e da aplicação do desconto padrão.
validarDependenciasServicos	Avalia as regras de dependência entre os serviços, verificando situações válidas e inválidas envolvendo Banho, Tosa e Hidratação.

11. Estrutura do Repositório

Caminho	Descrição
app.js	Arquivo principal da aplicação, responsável pela configuração do Express, das sessões, dos middlewares e pelo registro das rotas
database/init.js	Responsável pela criação das tabelas, execução das migrations e inserção dos dados iniciais do sistema
database/db.js	Configuração da conexão com o banco de dados SQLite
database/db-promise.js	Camada auxiliar que disponibiliza operações do banco utilizando Promises
routes/autenticacao.routes.js	Rotas relacionadas ao login, logout, recuperação e redefinição de senha
routes/clientes.routes.js	Operações de cadastro, edição, listagem e exclusão de clientes
routes/pets.routes.js	Operações relacionadas ao gerenciamento dos pets
routes/servicos.routes.js	Listagem dos serviços e atualização dos preços
routes/agendamentos.routes.js	Criação, edição, listagem, conclusão, exclusão e visualização dos agendamentos no calendário
routes/funcionarios.routes.js	Gerenciamento dos funcionários e usuários do sistema, com acesso restrito ao administrador
views/	Templates EJS organizados por módulos da aplicação
views/partials/sidebar.ejs	Componente de navegação reutilizado nas páginas internas
middlewares/autenticacao.js	Middlewares responsáveis pela autenticação e pela verificação de permissões de administrador

Caminho	Descrição
utils/helpers.js	Funções auxiliares utilizadas em validações, formatações e cálculos do sistema
public/css/	Arquivos CSS utilizados nas diferentes páginas da aplicação
testes/validacoes.test.js	Arquivo contendo os testes automatizados desenvolvidos com Jest
.github/workflows/ci-cd.yml	Configuração do pipeline de integração contínua utilizando GitHub Actions
render.yaml	Arquivo de configuração utilizado no processo de build e deploy da aplicação na plataforma Render

12. Como Executar o Projeto

12.1 Pré-requisitos

- Node.js 20 ou superior;
- npm (instalado junto com o Node.js);
- Git.

12.2 Instalação Local

1. Clonar o repositório:

git clone https://github.com/anttonio06/Desenvolvimento_web2.git

2. Acessar a pasta do projeto e instalar as dependências:

cd Desenvolvimento_web2

npm install

3. Inicializar o banco de dados: npm run init-db

4. Iniciar o servidor: npm start

5. Acessar no navegador: http://localhost:3000

12.3 Credenciais Padrão

Campo	Valor
E-mail	admin@petagenda.com
Senha	123456
Perfil	Administrador

12.4 Executar os Testes

Para executar os testes automatizados do projeto, utilize o comando: **npm test**

13. Dificuldades Encontradas

Dificuldade	Como foi resolvida
Gerenciamento de transações com a API baseada em callbacks do SQLite	Foi criado o arquivo db-promise.js, permitindo utilizar Promises nas operações com o banco e deixando o código assíncrono mais simples de manter.
Validação de conflito de horário com múltiplos serviços	A solução foi utilizar consultas que verificam pet, horário e serviços selecionados, permitindo detectar conflitos de agendamento envolvendo o mesmo pet e os mesmos serviços.
Regra de dependência entre serviços (Tosa e Hidratação exigem Banho)	Foi criada a função validarDependenciasServicos, utilizada para verificar se as combinações de serviços atendem às regras estabelecidas antes do registro do agendamento.
Cálculo do valor dos serviços de acordo com o porte do animal	Foi criada a função calcularValores, utilizada para determinar os valores dos serviços de acordo com o porte do animal.
Deploy da aplicação utilizando SQLite em ambiente de produção	Foi configurado o arquivo render.yaml e feita a integração com o GitHub Actions, permitindo automatizar o processo de deploy.
Persistência das sessões entre reinicializações do servidor	As sessões passaram a ser armazenadas em um arquivo separado com o auxílio do connect-sqlite3, evitando que usuários autenticados fossem desconectados após reinicializações do sistema.
Cadastro de funcionários e usuários de forma integrada	A criação dos registros nas tabelas usuarios e funcionarios foi realizada dentro de uma transação, mantendo a associação entre os dois registros.